- Integration tests allow you to test multiple components of your app at the same time, typically within the context of the ASP.NET Core framework itself. The Microsoft.AspNetCore.TestHost package provides a `TestServer` object that you can use to create a simple web host for testing. This creates an in-memory server that you can make requests to and receive responses from. You can use the `TestServer` directly when you wish to create integration tests for custom components like middleware.
- For more extensive integration tests of a real application, you should use the `WebApplicationFactory` class in the Microsoft.AspNetCore.Mvc.Testing package. Implement `IClassFixture<WebApplicationFactory<Program>>` on your test class, and inject an instance of `WebApplicationFactory<Program>` into the constructor. This creates an in-memory version of your whole app, using the same configuration, DI services, and middleware pipeline. You can send in-memory requests to your app to get the best idea of how your application will behave in production.
- To customize the `WebApplicationFactory`, call `WithWebHostBuilder()` and then call `ConfigureTestServices()`. This method is invoked after your app's standard DI configuration. This enables you to add or remove the default services for your app, such as to replace a class that contacts a third-party API with a stub implementation.